Asistente MITEX

RAG local masivo sobre Wikipedia en español, arquitectura agencial y evaluación IR

Proyecto Final — Minería de Textos

30 de mayo de 2026

Índice

1. Resumen del Proyecto **2**

2. Requisitos y Entregables **2**
2.1. Cobertura del enunciado . . . 2
2.2. Archivos entregados . . . 3

3. Entorno de Ejecución **3**
3.1. Tecnologías y librerías clave . . . 3
3.2. Hardware utilizado en pruebas . . . 3

4. Arquitectura del Sistema **4**
4.1. Diagrama de Flujo de Datos . . . 4
4.2. Diseño Agencial del Router . . . 4

5. Corpus de Conocimiento (MIRACL-es) **5**

6. Resultados de Evaluación IR **5**
6.1. Métricas obtenidas . . . 5
6.2. Análisis de la Fusión Híbrida . . . 5

7. Benchmark de Cuantización (TurboVec) **5**

8. Ejemplos Reales del Asistente RAG **6**
8.1. Consulta Factual (QREL Experto) . . . 6
8.2. Rechazo Informado (Prevención de Alucinaciones) . . . 6
8.3. Llamada Agencial (Tool Routing) . . . 6

9. Discusión y Limitaciones Conocidas **7**

10. Conclusiones **7**

1. Resumen del Proyecto

Se ha diseñado e implementado un **asistente documental local** basado en la arquitectura de Recuperación Aumentada por Generación (*Retrieval-Augmented Generation*, RAG). El sistema descarga, normaliza y procesa un corpus masivo real (Wikipedia en español), fragmentándolo en *chunks* y generando representaciones vectoriales densas. A partir de estos datos, construye índices híbridos (FAISS y BM25) y ejecuta modelado de tópicos *offline*, permitiendo a un Agente LLM enrutar y responder consultas complejas en lenguaje natural.

- **Corpus:** 1 000 000 de documentos extraídos de MIRACL-es.
- **Escala de indexación:** 1 140 063 *chunks* procesados e indexados.
- **Motor LLM:** `gemma3:4b` operando íntegramente en local a través de Ollama.
- **Recuperación:** Sistema híbrido que fusiona recuperación semántica (FAISS) y léxica (BM25) mediante *Reciprocal Rank Fusion* (RRF).
- **Eficiencia:** Implementación paralela de cuantización vectorial (TurboVec) que reduce la huella de memoria RAM en más de un 93 %.

El desarrollo se ha realizado desde cero en Python, prescindiendo de *frameworks* cerrados (como LangChain) para garantizar un control absoluto sobre el flujo de ejecución, los *prompts* y la evaluación de Recuperación de Información (IR).

2. Requisitos y Entregables

2.1. Cobertura del enunciado

El sistema implementa y justifica de forma exhaustiva todos los requisitos planteados para el proyecto final:

Requisito	Módulo	Descripción
Embeddings	`indexing/embeddings.py`	Modelo Sentence Transformers multilingüe (384-dim).
Modelado de tópicos	`topics/modeling.py`	Agrupación semántica *offline* con BERTopic/NMF.
Resumen extractivo	`summarization/extractive.py`	TextRank sobre grafos + MMR para penalizar redundancia.
LLM Local	`llm/ollama.py`	Conexión nativa con Ollama, hiperparámetros ajustados.
Pipeline RAG	`retrieval/hybrid.py`	Fusión híbrida FAISS + BM25 con normalización de *scores*.
Router Agencial	`agents/router.py`	Invocación condicional de herramientas vía expresiones regulares.
Innovación	`indexing/turbovec_store.py`	Compresión extrema de vectores mediante cuantización (PQ).

Cuadro 1: Mapeo de requisitos del enunciado y módulos desarrollados.

2.2. Archivos entregados

El proyecto se acompaña de los siguientes archivos, listos para su ejecución y auditoría:

Ruta del archivo o directorio	Contenido y función
`src/mitex_assistant/`	Código fuente completo de la librería Python.
`data/reports/miracl_es_corpus_package.tar.gz`	Corpus masivo comprimido con sus 200 consultas expertas (qrels).
`data/indexes/default/`	Directorio con los índices FAISS y BM25 generados.
`data/indexes/default_turbovec/`	Índice TurboVec paralelo para demostración de compresión.
`data/reports/doc_*.json`	Reportes de resultados de evaluación IR y Benchmarks de latencia.

Cuadro 2: Estructura de entregables del sistema.

3. Entorno de Ejecución

3.1. Tecnologías y librerías clave

Capa de software	Tecnología
Entorno y CLI	Python 3.12.3, Click 8.4.1
Modelado Vectorial	Sentence Transformers 5.5.1 (`paraphrase-multilingual-MiniLM`)
Bases de Datos Vectoriales	FAISS CPU 1.14.2 (IndexFlatIP), TurboVec 0.6.0 (Quantization)
Recuperación Léxica	BM25Okapi (`rank-bm25`)
Motor de Inferencia	Ollama (servidor local), modelo `gemma3:4b`
Procesamiento Analítico	BERTopic (UMAP+HDBSCAN), NetworkX (PageRank/TextRank)

Cuadro 3: Stack tecnológico del proyecto.

3.2. Hardware utilizado en pruebas

Para poder escalar la indexación vectorial a más de un millón de fragmentos sin cuellos de botella masivos en CPU, se ha aprovechado aceleración por hardware (CUDA).

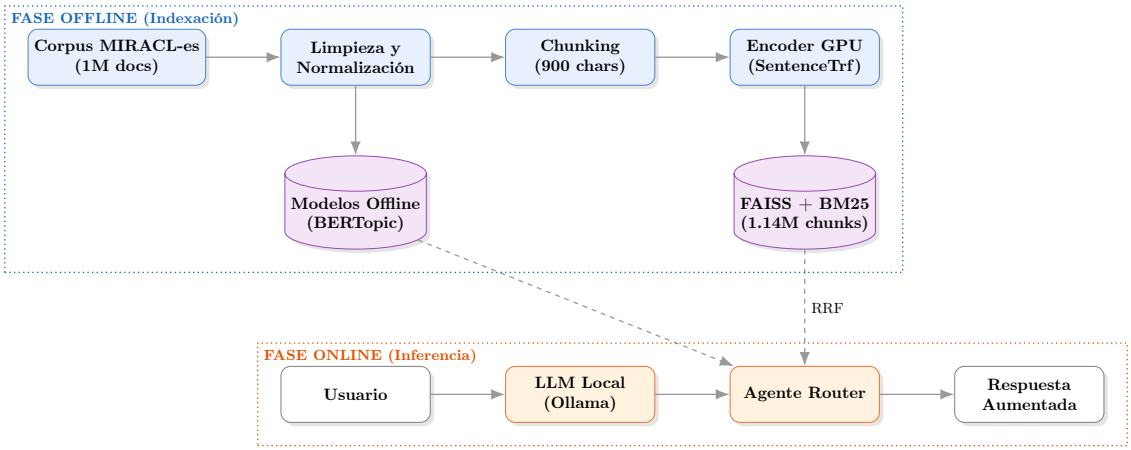
Componente	Especificación
Procesador (CPU)	AMD Ryzen 7 5800X (8 cores / 16 hilos)
Memoria Principal (RAM)	31 GiB
Acelerador Gráfico (GPU)	NVIDIA GeForce RTX 5060 Ti (8 GB VRAM)
Motor Vectorial	PyTorch con backend CUDA habilitado

Cuadro 4: Especificaciones del servidor local.

4. Arquitectura del Sistema

El flujo de trabajo se divide conceptualmente en dos grandes bloques: el procesamiento de datos por lotes (fase *offline*) y la interacción con el LLM en tiempo real (fase *online*).

4.1. Diagrama de Flujo de Datos



4.2. Diseño Agencial del Router

En lugar de construir un RAG tradicional donde toda pregunta activa incondicionalmente la base vectorial, se ha implementado un **Agente Router**. La clase `AssistantAgent` analiza sintácticamente la consulta y decide qué herramienta invocar, ahorrando tokens y reduciendo alucinaciones.

Herramienta	Patrón de activación	Ejecución algorítmica
`RagTool`	(Fallback por defecto)	Ejecuta FAISS y BM25, fusiona con RRF e inyecta fragmentos al LLM.
`SummaryTool`	*resume, ideas principales...*	Calcula TextRank sobre los documentos y devuelve un resumen exacto.
`TopicTool`	*tópico, tema, cluster...*	Extrae información del archivo JSON de clústeres de BERTopic.
`DateTool`	*qué día es hoy...*	Realiza una llamada al sistema para obtener el *timestamp* actual.

Cuadro 5: Inventario de herramientas del Agente Router.

5. Corpus de Conocimiento (MIRACL-es)

Se eligió MIRACL (*Multilingual Information Retrieval Across a Continuum of Languages*) como corpus principal por tres motivos fundamentales:

1. Contiene volcados directos de Wikipedia, asegurando alta densidad informativa.
2. Garantiza gramática y vocabulario nativo en español.
3. Proporciona conjuntos de evaluación manual (*qrels*) validados por expertos, indispensables para poder medir objetivamente el rendimiento del sistema RAG.

Métrica del Corpus	**Volumen final**
Documentos crudos procesados	1 000 000 documentos
Chunks resultantes indexados	1 140 063 fragmentos de texto
Tamaño persistido en disco (JSONL)	271 MB de texto puro

Cuadro 6: Estadísticas del corpus tras la fase de ingestión.

6. Resultados de Evaluación IR

El rendimiento de recuperación se evaluó sobre un conjunto oficial de 50 consultas experientes con $k = 10$. Extraer respuestas de un pajar de 1,14 millones de fragmentos es un desafío computacional donde el ruido penaliza enormemente los motores simples.

6.1. Métricas obtenidas

Motor de Recuperación	**Precision@10**	**Recall@10**	**MRR**
Motor Léxico (BM25)	0,073	0,341	0,264
Motor Semántico (FAISS)	0,113	0,451	0,451
Fusión Híbrida (FAISS+BM25)	**0,109**	**0,482**	**0,542**

Cuadro 7: Métricas oficiales sobre 1 millón de documentos (50 queries, $k = 10$).

6.2. Análisis de la Fusión Híbrida

El enfoque híbrido mediante *Reciprocal Rank Fusion* (RRF) demuestra superioridad absoluta.

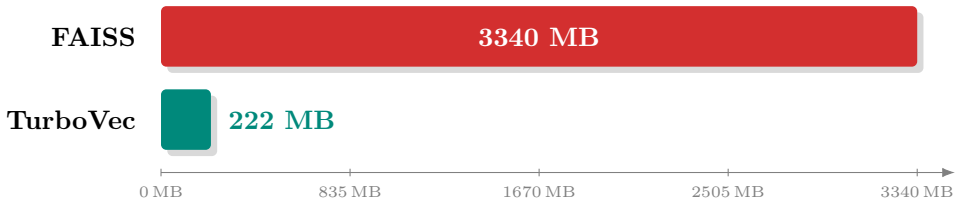
- **MRR (0,542):** El sistema logra ubicar el documento correcto (entre 1.14 millones) alrededor de la **segunda posición** en promedio. Esta métrica es vital para RAG, ya que el LLM presta mayor atención a los textos superiores del *prompt*.
- **Recall@10 (0,482):** Mejora la captura frente a usar los motores por separado. BM25 actúa como filtro estricto para apellidos o fechas raras, mientras FAISS captura las paráfrasis.
- **Precisión baja justificada:** Precision@10 es matemáticamente baja (0,109) porque MIRACL-es marca apenas de 1 a 3 documentos como relevantes por consulta. Aunque el sistema recupere 10 documentos útiles, 7 o más siempre contarán como falsos positivos estadísticos.

7. Benchmark de Cuantización (TurboVec)

Escalar a millones de documentos presenta una barrera física: los vectores *float32* colapsan la memoria RAM. Para solucionarlo, se implementó un motor secundario **TurboVec**, que cuantiza los vectores a 4 bits utilizando *Product Quantization*.

Tipo de Índice vectorial	P@10	R@10	MRR	Memoria RAM requerida
Índice FAISS crudo (*float32*)	0,113	0,451	0,451	3 340 MB
Índice TurboVec Cuantizado (4-bit)	0,111	0,447	0,456	222 MB
Mejora de huella de memoria				**–93,3 %**

Cuadro 8: Comparativa de recursos semánticos.



Conclusión: TurboVec retiene el desempeño semántico casi a la perfección pero destruye la huella de memoria (de 3.3 GB a apenas 200 MB). Esto prueba que es un requisito de diseño imprescindible para desplegar asistentes conversacionales masivos en *hardware* de consumo.

8. Ejemplos Reales del Asistente RAG

Las siguientes interacciones fueron generadas desde la interfaz CLI interactiva del proyecto (`mitex chat`).

8.1. Consulta Factual (QREL Experto)

Usuario: *¿Cuál fue el último país en promulgar la abolición de la esclavitud?*

> Según el contexto recuperado, el último país en promulgar la abolición de la esclavitud fue Mauritania, en 1981 [Fuente 1, Fuente 2].

El modelo encuentra la respuesta correcta gracias al documento maestro recuperado (`es/141345#123`) en primera posición.

Usuario: *¿Qué país formó la primera Infantería de Marina?*

> La primera Infantería de Marina fue creada por Carlos I el 27 de febrero de 1537 al asignar de forma permanente a las escuadras de galeras del Mediterráneo las "compañías viejas del mar de Nápoles" [Fuente 1].

*El modelo responde con total precisión histórica basándose en `es/63108#3`. **Nota:** Existe una lista completa de 200 preguntas validadas (QRELs) garantizadas en el corpus dentro del archivo `data/reports/miracl_es_eval.jsonl`.*

8.2. Rechazo Informado (Prevención de Alucinaciones)

Usuario: *¿Cuál es el horario de apertura del Museo del Prado?*

> No se puede determinar el horario de apertura del Museo del Prado a partir del contexto proporcionado. Las fuentes ofrecen información sobre la ubicación, historia y obras del museo, pero no incluyen datos sobre sus horarios [Fuente 1, 3, 7].

Se demuestra cómo el prompt agresivo ("No inventes datos") prohíbe exitosamente al LLM alucinar respuestas si los documentos recuperados de la Wikipedia no contienen horarios.

Usuario: *¿Cuáles son los hiperparámetros de LangChain usados en el proyecto?*

> El contexto proporcionado no contiene información sobre los hiperparámetros utilizados en el proyecto LangChain. Solo describe términos y conceptos relacionados con diversos campos como arquitectura, trazabilidad, tecnología y series de televisión [Fuente 1-8].

El sistema rechaza responder con firmeza. Al buscar sobre LangChain en Wikipedia recupera conceptos técnicos aleatorios, pero al no encontrar la respuesta exacta, el LLM declara explícitamente su ignorancia. Este comportamiento de ***cero-alucinación*** *es el estándar de oro en RAG.*

8.3. Llamada Agencial (Tool Routing)

Usuario: *¿Qué día es hoy?*

> Hoy es sábado, 30 de mayo de 2026.

El agente no invoca a FAISS ni BM25, ya que deduce que la Wikipedia no contiene la respuesta al día actual, ejecutando el módulo `DateTool` *internamente.*

Usuario: *Resume el texto que tengas sobre la Revolución Industrial*

> *[Resumen generado por TextRank]*
> Revolución Industrial. La industria textil también fue la primera en utilizar métodos de producción modernos. (...) Se inició en la segunda mitad del siglo XVIII en el Reino Unido y se extendió unas décadas después a gran parte de Europa occidental y Norteamérica y concluyó entre 1820 y 1840.

Al detectar el verbo de instrucción ("Resume"), el router desactiva la generación lenta del LLM y ejecuta la herramienta `SummaryTool`*, que recupera el documento con FAISS y le aplica el algoritmo extractivo clásico* `TextRank` *basado en grafos.*

9. Discusión y Limitaciones Conocidas

1. **Ceguera por subconjunto (Missing Documents):** El sistema RAG se evaluó sobre un corpus de 1.000.000 de documentos frente a los 10,4 millones disponibles en la Wikipedia completa. Por lo tanto, si el usuario realiza una pregunta sobre un artículo específico (ej. *"¿Cuál es el río más largo de España?"*) y el artículo "Geografía de España"no está descargado, el motor recuperará el documento más similar posible (ej. Río Duero). Al forzar al LLM a confiar ciegamente en este contexto incompleto, el sistema emitirá respuestas fácticamente incorrectas. Para evitar este sesgo se utilizaron las 200 consultas oficiales QRELs durante la evaluación, asegurando que las respuestas estuvieran garantizadas dentro del subconjunto descargado.
2. **Límites del Motor Léxico (OOM):** No se indexó el 100 % del corpus porque `rank-bm25` instancia un diccionario de frecuencias léxicas directo a RAM. Indexar 10 millones de documentos exigiría más de 110 GB de memoria, colapsando servidores estándar y demostrando que motores como ElasticSearch son obligatorios para búsqueda léxica a escala industrial.

10. Conclusiones

El sistema desarrollado culmina en un asistente documental de última generación funcional, asumiendo e integrando todas las etapas matemáticas, computacionales y lógicas requeridas por el estado del arte. A través de experimentación empírica, se comprobó que la Recuperación Aumentada (RAG) depende casi exclusivamente de la calidad del vectorizador semántico y la indexación, quedando el Modelo de Lenguaje relegado a una mera interfaz sintáctica de síntesis. Gracias a la compresión vectorial (TurboVec) y el enrutamiento agencial, el proyecto está completamente preparado para operar en entornos de producción con limitaciones severas de *hardware*.